

Mirage Persistent Kernel: 面向张量程序 Mega-Kernel 化的编译器与运行时

Xinhao Cheng^{1,*} Zhihao Zhang^{1,*} Yu Zhou^{1,*} Jianan Ji^{1,*} Jinchen Jiang² Zepeng Zhao¹
Ziruo Xiao¹ Zihao Ye³ Yingyi Huang³ Ruihang Lai¹ Hongyi Jin¹ Bohan Hou¹
Mengdi Wu¹ Yixin Dong¹ Anthony Yip¹ Zihao Ye⁴ Songting Wang¹ Wenqin Yang⁵
Xupeng Miao⁶ Tianqi Chen^{1,3} Zhihao Jia¹

卡内基梅隆大学¹ 清华大学² NVIDIA³
密歇根大学⁴ 独立研究者⁵ 普渡大学⁶

Abstract

我们提出 Mirage Persistent Kernel (MPK), 这是首个能够将多 GPU 模型推理自动转换为单个高性能 mega-kernel 的编译器与运行时系统。MPK 引入 SM 级图表示, 以单个流式多处理器 (SM) 为粒度刻画数据依赖, 从而支持跨算子软件流水、细粒度内核重叠等以往难以实现的 GPU 优化。MPK 编译器将张量程序降为高度优化的 SM 级任务图, 并为各个任务生成优化后的 CUDA 实现; MPK 的核内并行运行时则在单个 mega-kernel 内, 以跨 SM 的分布式调度方式执行这些任务。两者结合后, 只需极少的开发工作便可实现端到端内核融合, 同时保留现有编程模型的灵活性。实验表明, MPK 可将端到端推理延迟最高降低 1.7 $\times$, 显著优于现有按算子逐核执行的 LLM 服务系统, 并将性能进一步逼近硬件极限。MPK 已在 <https://github.com/mirage-project/mirage> 开源。

1 引言

在 GPU 上实现高性能机器学习推理, 对现代 AI 应用至关重要, 因为推理延迟会直接影响可用性与成本。如今的大多数 ML 系统都会将模型计算表示为张量程序, 也就是一个有向无环图 (DAG): 图中的节点表示张量代数算子, 例如矩阵乘法; 边则表示这些算子消费和产生的张量, 即 n 维数组。

现有系统通常为每个算子单独启动一个 GPU kernel。这样的 kernel 要么由专家手工优化 [16, 34], 要么由 ML 编译器自动生成 [14, 28, 32]。然而, 这种按算子逐核执行 (kernel-per-operator) 的模型, 会限制一系列关键的跨算子 GPU 优化。

首先, 现代 GPU 会在同一 stream 上相邻的两个 kernel 启动之间隐式插入 *kernel barrier*, 确保前一个 kernel 的所有线程都结束之后, 下一个 kernel 才能开始执行。这个机制虽然保证了依赖关系的正确性, 但也阻断了跨算子软件流水, 使得有依赖关系的算子只能严格串行执行。NVIDIA 最近提出的 *programmatically dependent launch* (PDL) [10] 允许同一 stream 上的 kernel 出现一定程度

的重叠, 但它需要显著改变 kernel 的结构与控制流, 工程代价很高。

其次, 按算子逐核执行还会阻碍细粒度的计算-通信重叠。因为系统只在算子粒度上记录依赖, 所以运行时往往需要等整个算子完成之后, 才能启动后续的通信或计算。举例来说, 如果矩阵乘法之后接着一个 all-reduce, 而它们被实现成两个独立 kernel, 那么 all-reduce 必须等待整个矩阵乘法结束, 即便它实际上只依赖其中一部分输出。要利用这类机会, 就必须以比单个 kernel 更细的粒度刻画和执行数据依赖。

最后, 这种执行方式通常要求每次推理迭代发起数百到上千个 kernel。为了降低启动开销, 当前系统高度依赖 *CUDA Graphs*, 用以捕获一串 GPU 操作, 并以较低的开销重新发射。但 *CUDA Graphs* 本质上仍然偏静态: 只要控制流、张量形状或数据依赖发生变化, 就往往需要重新实例化或修改整张图, 这对于模型推理中常见的动态工作负载并不友好。

一种有前景的替代思路, 是将整个模型的计算与通信都融合到单个 *mega-kernel* (也称 *persistent kernel*) 中。在这种设计下, 系统只启动一次 GPU kernel, 随后该 kernel 持续执行整个模型, 包括层内计算与跨 GPU 通信。

mega-kernel 可以系统性地缓解按算子逐核执行的局限。其一, 它通过避免反复发射 kernel, 直接消除了启动开销。其二, 当多个算子都被放入同一个 kernel 后, 系统就可以执行跨算子软件流水, 例如在当前算子仍在计算时, 预取下一个算子所需的数据。其三, 它还能支持计算与 GPU 间通信的细粒度重叠, 更有效地隐藏通信延迟。

但要把机器学习模型自动转换成高性能 *mega-kernel* 仍然非常困难。当前主流 ML 系统, 如 PyTorch [24]、Triton [28] 与 TVM [14], 都不支持端到端的 *mega-kernel* 生成。与此同时, 现有系统还依赖一个碎片化的专用库生态: 通信依赖 NCCL [4] 或 NVSHMEM [9], 注意力依赖 FlashInfer [34] 或 FlashAttention [16], 其余自定义算子再交给 CUDA 或 Triton。这种分裂的实现方式, 使整个推理流水线很难统一进单个 kernel。

为了解决这一问题, 我们提出 *Mirage Persistent Kernel* (MPK), 这是首个能够将多 GPU 模型推理自动转换为高性能 *mega-kernel* 的编译器与运行时系统。对于用户而

* 同等贡献。

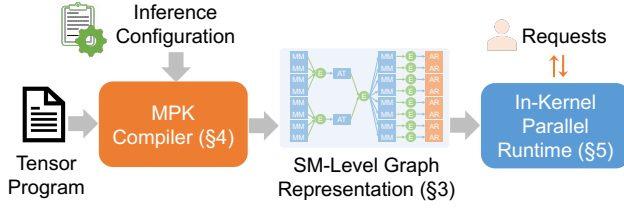


Figure 1: MPK 总览。

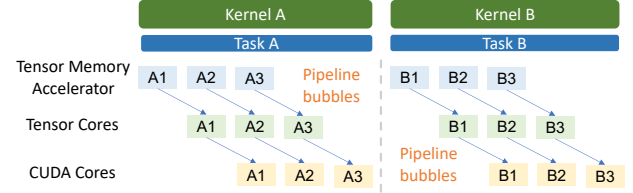
言，只需在 PyTorch 中做少量代码改动，就能把模型编译成一个 mega-kernel；相较于普通 PyTorch 配合 CUDA Graphs 或 `torch.compile` 的做法，MPK 可以带来显著的性能收益。它将 mega-kernel 的性能优势与现有 ML 框架的可用性结合在了一起。

MPK 的核心思想，是不再以整个 GPU，而是以单个流式多处理器（SM）为粒度表示计算与跨 GPU 通信。为此，MPK 引入了称为 *tGraph* 的 SM 级图表示：图中的节点是运行在各个 SM 上的任务，边则刻画任务之间的细粒度依赖。这样的表示能够暴露更多并行性，并支持跨算子软件流水和细粒度 kernel 重叠等传统按算子逐核执行模型无法实现的优化。Figure 1 展示了支撑这一思路的两个关键组件。

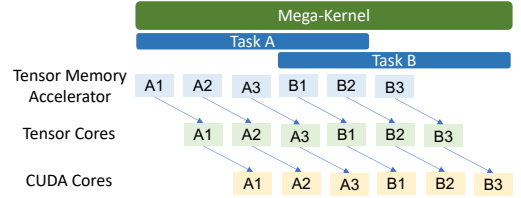
MPK 编译器。 MPK 编译器以张量程序和推理配置为输入，自动将原始计算图转化为面向特定推理配置与 GPU 架构的高性能 SM 级 *tGraph*。编译过程中，系统会执行事件融合、图规范化与图线性化等一系列优化，以降低同步开销并提升生成任务图的执行效率。此外，MPK 还会基于已有的超级优化技术 [32]，自动为每个任务生成高性能 CUDA 实现，从而保证 SM 级执行同样高效。

核内并行运行时。 MPK 通过一个完全嵌入在 mega-kernel 内部的并行运行时来执行 SM 级 *tGraph*，从而在模型执行期间不再需要任何额外的 kernel 启动，并能以细粒度方式控制任务调度与执行。为实现这一点，运行时将 GPU 的 SM 划分为 *worker* 与 *scheduler* 两类角色。*worker* 维护本地任务队列，并按先进先出的顺序执行任务；*scheduler* 维护任务依赖，并在条件满足时将任务分发出去。整个运行时采用事件驱动、完全异步的执行模型，以最大化 GPU 利用率。同时，MPK 结合即时启动与提前启动两种方式，形成混合任务发射策略，以在低开销和动态负载均衡之间取得平衡。

实验结果。 我们将 MPK 实现为 PyTorch 的一个 kernel backend。用户只需修改少量代码，就可以把 PyTorch 程序编译成 MPK mega-kernel。我们在五个常见模型上，跨三代 NVIDIA GPU (A100、H100 和 B200) 进行了评测。即便面对像 SGLang 和 vLLM 这类已经广泛部署并经过深度优化的按算子逐核执行系统，MPK 在单 GPU 与多 GPU 部署中仍然取得了 1.0–1.7× 的性能提升，并将 LLM 推理性能进一步推向硬件极限。



(a) Kernel barrier 会阻断跨任务流水。



(b) MPK 同时支持任务内与任务间流水。

Figure 2: 比较 MPK 与现有方法在任务内和任务间流水支持上的差异。

2 背景

本节首先回顾以 kernel 为中心的 GPU 编程模型及其局限 (§ 2.1)，随后介绍 kernel fusion 与 mega-kernel 技术 (§ 2.2)，它们共同构成了 MPK 的设计动机。

2.1 GPU 编程模型

在 GPU 上，计算通常以 *kernel* 的形式组织。每个 kernel 都是在大量核心上以 SPMD (single-program, multiple-data) 方式并发执行的函数。一个 kernel 由若干 *thread block* 组成，每个 block 会被分配到一个流式多处理器（SM）上执行；block 内包含多个 *thread*，分别处理不同的数据元素。每个线程拥有私有寄存器文件，而同一 block 内的线程则可通过低延迟的 *shared memory* 交换数据并完成协作。kernel 的输入与输出均存储在 GPU 的 *device memory* 中。

CUDA 编程模型并不支持在同一个 kernel 内对不同 *thread block* 进行直接同步，因为现代 GPU 架构缺乏相应的硬件协同机制。因此，跨算子依赖（例如一个矩阵乘法必须在另一个开始前完成）通常依赖 *kernel barrier* 来保证，这些 barrier 由 GPU 运行时自动插入在同一 stream 上相邻启动的 kernel 之间。

kernel barrier 虽然简化了依赖管理，但也会阻碍一些关键 GPU 优化，例如跨 kernel 软件流水与细粒度 kernel 重叠。

软件流水。 现代 GPU 正变得越来越异构，会集成 tensor core、tensor memory accelerator (TMA) 等专用部件。由于 TMA 的 load/store 指令是异步执行的，因此数据搬运可以与 tensor core 和 CUDA core 上的计算并行进行。若想充分发挥这些硬件单元的能力，就需要软件流水，即把不同迭代中的数据搬运与计算阶段交错执行，以提高整体硬件利用率。

现有系统通常只能实现任务内流水，如 Figure 2a 所

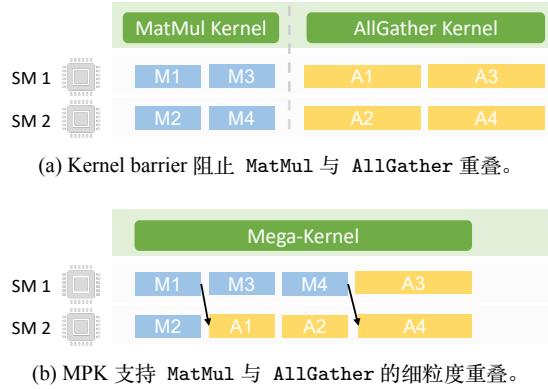


Figure 3: 比较 MPK 与现有方法在任务间细粒度 kernel 重叠上的能力。Figure 3b 中的黑色箭头表示保证正确性的依赖关系。

示：一个任务被拆成多个迭代，不同迭代中的数据传输、矩阵计算与辅助操作分别在流水线的不同阶段执行。但 kernel barrier 会把流水限制在单个任务内部，阻止任务间流水，从而在任务切换处产生流水气泡，导致硬件资源利用不足。

细粒度 kernel 重叠。 kernel barrier 还会阻碍那些使用不同硬件资源的 kernel 之间进行重叠，例如计算与通信。原因在于它只能在整个 kernel 粒度上施加依赖，而不是在数据块粒度上施加依赖。Figure 3a 给出了 LLM 中的常见模式：MatMul 之后紧跟一个 AllGather。在现有系统里，它们通常被实现为两个独立 kernel，因此 AllGather 的所有 thread block 都必须等前一个 MatMul kernel 的所有 thread block 完成后才能开始。

但在实际中，AllGather 与 MatMul 之间的数据依赖远比这更细。由于 AllGather 本质上是逐元素通信，它的每个 thread block 往往只依赖某一个 MatMul thread block 的输出。这样的依赖结构天然支持细粒度 kernel 重叠：只要依赖得到满足，不同 SM 就可以并行执行 MatMul 与 AllGather。先前工作已经指出，这样的重叠能够同时利用现代 GPU 的计算能力和通信带宽 [40]。但要做到这一点，就必须在 sub-kernel 粒度上实现跨 SM 同步，而传统的 kernel barrier 并不具备这种能力。

2.2 Kernel Fusion

Kernel fusion 通过把多个本来需要顺序执行的 GPU kernel 合并成一个语义等价的单一 kernel，来消除 kernel barrier。这样做可以避免中间结果落回 device memory，减少显存访问，并消除 kernel 启动开销。

Kernel fusion 已广泛应用于张量程序编译器中。PyTorch、JAX 和 TVM 等框架依靠规则或启发式方法融合相邻 kernel [13, 14, 24]；Mirage 与 TASO 这类系统则利用超级优化自动搜索融合规则 [21, 32]。不过，现有编译器通常只能融合局部、规模较小的一组算子，因为为复杂张量程序生成一个单一且等价的 kernel 在计算上极为困难，很多时候甚至不可行。

mega-kernel 则把 kernel fusion 推向极致：它把一个张

量程序中的全部计算与通信都融合到同一个持久 kernel 中，并通过 device memory 中的同步原语协调不同 SM 的执行。尽管这种范式具有明显的性能优势，PyTorch、Triton、JAX 和 TVM 等现有 ML 编译器仍不支持 mega-kernel 编译。当前能见到的 mega-kernel，大多仍是 GPU 专家针对特定模型手工打造的。

例如，FlashDMoE 将 MoE 计算与跨 GPU 通信融合进单个 kernel [12]；Spector 等人则为 LLAMA-1B 手工设计并实现了一个低延迟 mega-kernel [8, 29]。这类方案都需要很高的工程投入与深入的 GPU 专业知识。相比之下，MPK 采用编译器驱动的方法，能够自动把张量程序转换为优化后的 mega-kernel，从而消除人工 mega-kernel 化的负担。

3 SM 级图表示

本节介绍 *tGraph*，这是一种以单个流式多处理器 (SM) 为粒度来表示张量程序计算的图结构。这样的细粒度表示能暴露出更多并行性，并支持跨算子软件流水与细粒度 kernel 重叠等优化，而这些能力在传统按算子逐核执行的模型中无法获得。

Figure 4 给出了一个 *tGraph* 示例。图中的每个节点要么是 *task*，要么是 *event*。蓝色（或橙色）矩形表示 *task*，对应单个 SM 上执行的一个计算或通信单元；绿色圆点表示 *event*，用来完成任务之间的同步。任务和事件在图中交替出现：每个任务只会向外连接到它的 *triggering event*，并从若干 *dependent event* 接收入边。当一个任务依赖的事件全部 *activated* 后，该任务即可执行；任务执行完毕后，又会通知自己的触发事件。一个事件在收到与之关联的全部任务通知后才会被激活。

这种结构以比传统计算图更细的粒度描述依赖关系。以多 GPU LLM serving 为例，经常会出现 MatMul 之后接 AllReduce 的模式 (Figure 4a)。在现有系统中，由于 kernel barrier 只能在整个 kernel 粒度上同步，这两个算子通常只能顺序执行。相比之下，SM 级任务图能够表达更精确的任务级依赖：因为 AllReduce 是按元素通信的，每个 AllReduce 任务实际上只依赖一个对应的 MatMul 任务。通过在任务对之间插入细粒度事件，MPK 可以在保持正确性的同时，让计算密集的 MatMul 任务与通信密集的 AllReduce 任务重叠执行，从而最大化 GPU 利用率。

对于同一个计算图，往往存在多个不同的 *tGraph* 表示。Figure 4c 展示了一个可行但次优的版本：其中的事件只表达算子级依赖，效果与传统 kernel barrier 相似。§ 4 将进一步说明 MPK 如何通过推断精确的数据依赖，生成高性能任务图，从而最大化并发度并最小化同步开销。

与 CUDA Graphs 的比较。 *tGraphs* 可以看作是 CUDA Graphs 的一个更底层扩展。两者都以静态方式实例化，并显式编码操作之间的依赖关系。但 CUDA Graphs 只在 kernel 粒度上描述依赖，而 *tGraphs* 则下探到单个 SM 任务与 sub-kernel 事件的粒度。CUDA Graphs 的主要作用是描述 kernel 的发射顺序，并依赖 stream 语义完成同

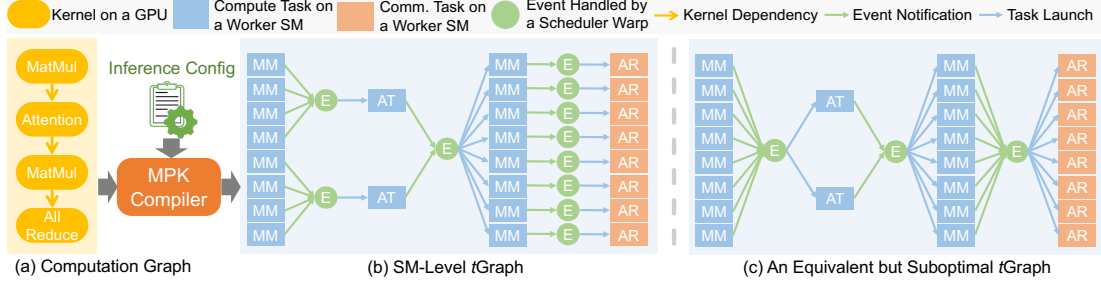


Figure 4: MPK 编译器将 kernel 级计算图转换为优化后的 SM 级 $tGraph$ 。图中的 MM、AT 和 AR 分别表示 MatMul、Attention 与 AllReduce 任务。

步，因此它的融合与重叠能力都受限于 kernel 边界。与之不同， $tGraphs$ 会显式建模算子内部和算子之间的依赖，从而支持跨 SM 的细粒度同步，并在单个 kernel 内实现计算与通信的重叠。这使得 MPK 能挖掘出 CUDA Graphs 以及传统 kernel 级执行模型无法利用的并行性。

4 MPK 编译器

本节介绍 MPK 编译器。它以计算图及其对应的推理配置为输入，生成同时针对目标配置与底层 GPU 架构优化的 $tGraph$ 。整体编译流程如 Figure 5 所示。

4.1 $tGraph$ 生成

算子分解。 MPK 编译器通过对算子输出张量进行切分，把输入计算图中的每个算子分解成一组任务，使不同任务负责输出中彼此不重叠的区域，并可以在多个 SM 上并行执行。多数张量代数算子都可以沿多个输出维度切分。例如，一个矩阵乘法的输出既可以沿行方向切块，也可以沿列方向切块，以暴露更多并行性。

切分策略的性能同时依赖问题形状与目标 GPU 架构。为了找到高效方案，MPK 优先选择那些能最小化从 device memory 向 shared memory 读入数据量的切分方式，因为访问 device memory 的代价显著高于访问 shared memory，且也高于在 CUDA core 或 tensor core 上执行计算。默认情况下，MPK 生成的任务数量会与 SM 数量成比例，以促进执行阶段的负载均衡。系统也向用户暴露接口，允许按输出维度指定并行度，从而自定义切分策略。

依赖分析。 MPK 使用 *event* 来表示任务之间的依赖。对于共享同一张量的两个算子，MPK 会枚举它们之间所有任务对，并仅在任务 t_1 产出的输出区域与任务 t_2 消费的输入区域发生重叠时，为该任务对 (t_1, t_2) 引入一个事件 e 。这一事件表示：只有当 t_1 完成并产出所需数据后， t_2 才能开始。为此，MPK 会在 $tGraph$ 中插入两条边 (t_1, e) 与 (e, t_2) 。这种细粒度依赖分析既完整保留了 producer-consumer 关系，又尽可能暴露出彼此独立任务之间的并行性。

事件融合。 MPK 进一步使用两类互补的事件融合来消除冗余同步点，并简化构造出的 $tGraph$ ：后继集合融合

与前驱集合融合。对于任一事件 e ，记 $InTasks(e)$ 为触发该事件的任务集合， $OutTasks(e)$ 为依赖该事件的任务集合。借助这两个集合，我们可以判断不同事件是否具有相同依赖结构，从而合并。

第一类是后继集合融合。如果多个事件面向同一组后继任务，那么这些任务只有在所有相关事件都激活后才能执行，把它们分开表示并不会增加任何调度自由度。

Definition 4.1 (后继集合融合). 对于 $tGraph$ 中任意两个事件 e_1 和 e_2 ，当且仅当 $OutTasks(e_1) = OutTasks(e_2)$ 时，可应用后继集合融合。MPK 删除 e_1 与 e_2 ，并引入新的融合事件 e' ，其中 $InTasks(e') = InTasks(e_1) \cup InTasks(e_2)$ ，且 $OutTasks(e') = OutTasks(e_1)$ 。

例如，Figure 5(b) 中的 e_{10} 与 e_{14} 都是任务 O_1 的前置条件，因此可以被融合成 Figure 5(c) 中的一个新事件。

第二类是前驱集合融合。如果多个事件依赖完全相同的一组前驱任务，那么它们会被同时触发，继续保留成多个同步节点只会增加图复杂度。

Definition 4.2 (前驱集合融合). 对于 $tGraph \mathcal{T}$ 中任意两个事件 e_1 和 e_2 ，当且仅当 $InTasks(e_1) = InTasks(e_2)$ 时，可应用前驱集合融合。MPK 删除 e_1 与 e_2 ，并引入新的融合事件 e' ，其中 $InTasks(e') = InTasks(e_1)$ ，且 $OutTasks(e') = OutTasks(e_1) \cup OutTasks(e_2)$ 。

$tGraph$ 规范化。 经过事件融合后，运行时仍然会面临两个实现问题：一是某个任务可能同时依赖多个事件或触发多个事件；二是一个事件可能会触发大量任务，导致编码成本较高。MPK 首先通过 $tGraph$ 规范化解决第一个问题。规范化会把输入 $tGraph$ 转换为一个功能等价、但满足每个任务至多只有一个依赖事件和一个触发事件的形式。

为此，MPK 使用两种变换来把每个任务的 fan-in 和 fan-out 都降低到 1。若某个任务 T_0 会触发多个事件 $e_1, \dots, e_k$ ，则系统引入一个新事件 e' 以及 k 个空任务 $T_1, \dots, T_k$ 。这样一来， T_0 只触发 e' ，而每个空任务 T_i 再分别触发原来的一个事件 e_i 。相反，如果某个任务 T_0 依赖多个事件 $e_1, \dots, e_k$ ，则系统同样引入一个新事件 e' 与 k 个空任务，让 T_0 只依赖其中一个事件，而新增任务负责把其他事件汇聚到 e' 上。

这种规范化会额外引入少量任务与事件，但我们观察到其开销通常极小，在实验中始终低于 1%。原因是现

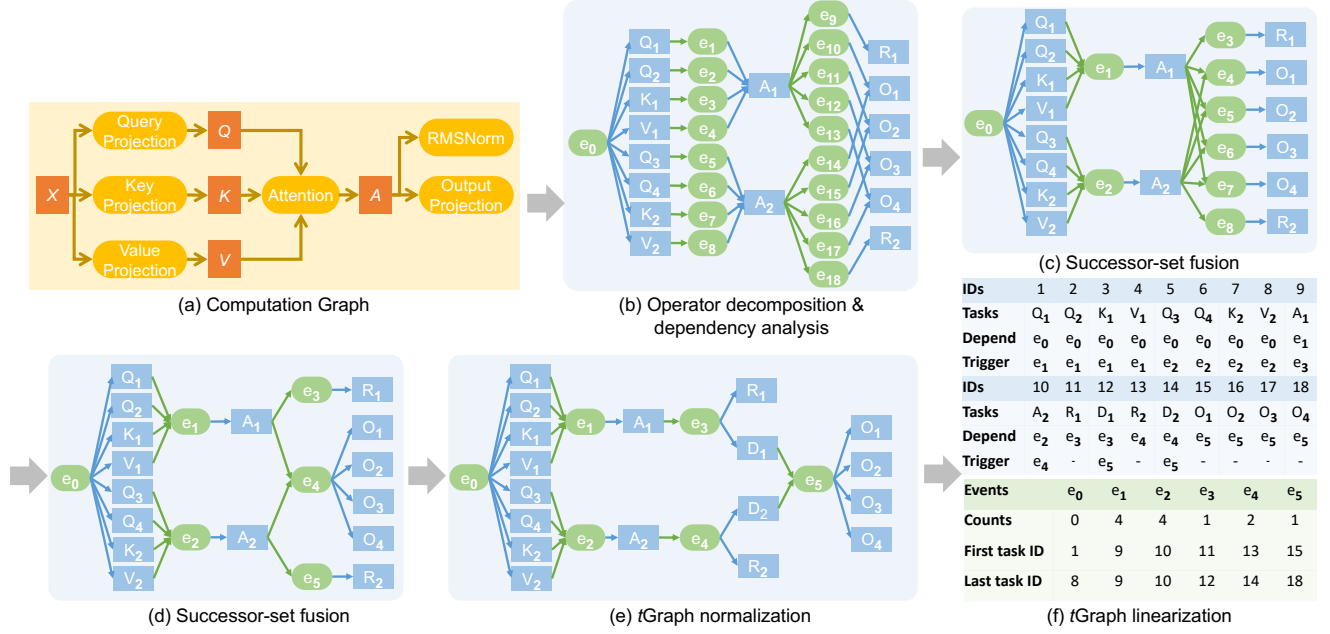


Figure 5: MPK 编译器工作流。(b) 中的 Q 、 K 、 V 、 A 、 O 和 R 分别表示 query projection、key projection、value projection、attention、output projection 与 RMSNorm 被分解后得到的任务集合。(e) 中的 D_1 与 D_2 是在 tGraph 规范化阶段插入的哑任务，用于保证每个任务都只有一个触发事件。最后，(f) 展示了 MPK 如何对 tGraph 进行线性化，并将任务与事件以统一的规范格式存储在 GPU 内存中。

实模型往往更偏向“深”而非“宽”，也就是顺序层较多，而可并行分叉的结构相对有限。

Algorithm 1 MPK 的 tGraph 线性化算法。该算法保证每个任务只会入队到 T 一次，每个事件只会入队到 E 一次。第 5–7 行保证同一事件触发的任务在 T 中连续出现。

输入: 一个规范化后的 tGraph $\mathcal{G}$

输出: 任务列表 T ，满足对任意事件 $e \in \mathcal{G}$ ，由 e 触发的任务在 T 中占据连续区间。

```

1:  $T \leftarrow \emptyset$ 
2:  $E \leftarrow \{e \in \mathcal{G} | e.counts = 0\}$     ▷ 将所有没有前驱任务的事件入队
3: while  $E$  非空 do
4:    $e \leftarrow E.dequeue()$ 
5:   for all task  $t \in \mathcal{G}$  do
6:     if  $t.dependent\_event = e$  then
7:        $T.enqueue(t)$ 
8:        $e' \leftarrow t.trigger\_event$ 
9:       if all tasks triggering  $e'$  are in  $T$  then
10:         $E.enqueue(e')$ 
11:
12: return  $T$ 

```

tGraph 线性化。 tGraph 规范化并不能解决第二个问题：即使完成融合后，一个事件仍可能需要触发很多任务，例如 Figure 5(e) 中的 e_5 需要触发 4 个任务。为避免显式存储这些任务索引列表，MPK 使用基于广度优先搜索 (BFS) 的线性化算法 (Algorithm 1) 来重排任务顺序。线性化的目标是保证由同一个事件触发的任务在最终顺序中占据一个连续区间。这样一来，一个事件的 fan-out 只需记录起始和结束任务索引即可，既保留了全

部依赖语义，又显著降低了存储成本。

Figure 5(f) 展示了线性化后 tGraph 在 GPU device memory 中的存储方式。对每个任务，MPK 仅记录其依赖事件与触发事件的索引；对每个事件，则记录其被激活所需的触发次数，以及它对应任务区间的首尾索引。当事件被激活后，运行时即可一次性发射该区间内的所有任务。

4.2 任务实现生成

除了构造 tGraph 之外，MPK 还需要为每个任务生成在单个 GPU SM 上执行的 device function。为此，MPK 借鉴已有工作，采用编译器超级优化的方式自动生成高性能实现。

具体而言，MPK 把超级优化的粒度下沉到 thread block，而不是整个 kernel。每个计算任务都配有一个参考 PyTorch 实现，MPK 利用 Mirage superoptimizer [32] 自动搜索最优的 thread block graph，再由 Mirage 编译器生成 CUDA 实现。生成结果会同时包含各种 SM 内优化，例如软件流水、寄存器复用与布局优化，以降低 bank conflict 并提升执行效率。

5 核内并行运行时

MPK 采用核内并行运行时，在单个 mega-kernel 内跨全部 SM 执行 tGraph。这样的设计既消除了 kernel 发射开销，也使系统能够细粒度控制调度、同步与执行顺序。

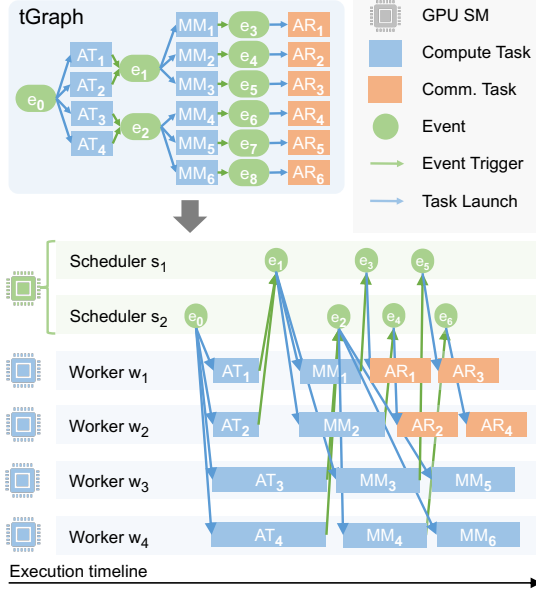


Figure 6: MPK 的事件驱动执行模型。圆形表示事件，蓝色（或橙色）矩形分别表示计算任务（或通信任务）。从事件指向任务的边表示任务发射，从任务指向事件的边表示该任务会触发对应事件。图中的 AT、MM 与 AR 分别表示 attention、matrix multiplication 与 AllReduce。

mega-kernel 一旦启动，就会持续管理计算与通信，直到一次推理工作负载结束。

为了支撑这一执行模式，MPK 会把 GPU 的 SM 划分为 *worker* 和 *scheduler*。每个 worker 运行在一个物理 SM 上，并维护独立的 *task queue*。worker 会不断从队列中取出任务，执行相应的计算或通信，并在任务完成后通知其触发事件。这样的设计既保证了 worker 的高利用率，也支持跨算子的异步执行。

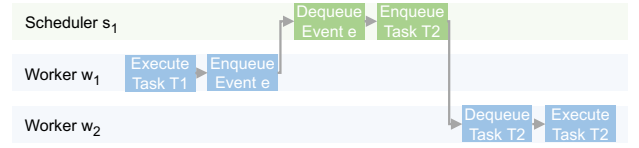
scheduler 以 *warp* 为粒度组织，每个 SM 上驻留 4 个 scheduler warp。每个 scheduler 维护一个事件队列，并持续轮询是否有新激活的事件；一旦发现，就把对应任务分发给 worker。worker 与 scheduler 的数量在 kernel 发射时就固定下来，并与 GPU 的物理 SM 数量匹配，从而避免在 kernel 内部进行动态角色切换。

本节余下部分将详细介绍这一核内运行时架构。§ 5.1 描述 MPK 的事件驱动执行模型；§ 5.2 比较两种互补的任务发射机制并说明 MPK 如何组合它们；§ 5.3 则介绍若干进一步降低运行时开销的系统优化。

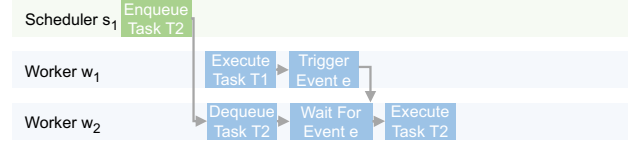
5.1 事件驱动执行

MPK 使用事件驱动模型执行 *tGraph*。每个 *tGraph* 都有一个没有任何前置条件的起始事件，例如 Figure 6 中的 e_0 。该事件最初会被放入某个 scheduler 的事件队列。当 scheduler 取出该事件后，它会发射所有依赖该事件的任务，例如 $AT_1, \dots, AT_4$ 。这些任务被分发给 worker 执行，任务结束后，worker 再去通知与之关联的触发事件。

当一个事件的所有前置条件都已完成，并且它收到足够次数的触发后，该事件就会被激活。一旦激活，它就



(a) 即时 (JIT) 任务发射。



(b) 提前 (AOT) 任务发射。

Figure 7: JIT 与 AOT 两种任务发射方式的比较。

会再次进入某个 scheduler 的事件队列，驱动运行时继续沿着 *tGraph* 向前推进。换言之，在 MPK 中，事件是驱动任务执行的核心机制，这使得系统可以在细粒度上实现完全异步的执行。

5.2 混合任务发射

一个任务可以通过两种方式被放入 worker 的任务队列：*just-in-time* (JIT) 或 *ahead-of-time* (AOT)。在 JIT 模式下，scheduler 只有在任务的依赖事件完全激活后，才会把该任务分配给某个 worker，因此任务一旦入队就可以立刻执行。相反，在 AOT 模式下，运行时会在前驱事件尚未激活之前，就把任务预先放入 worker 的本地队列；worker 会在本地等待依赖满足后再执行。

这两种方式各有优势。JIT 更利于应对负载不均衡，因为 scheduler 能在依赖满足的瞬间根据当前系统状态选择空闲 worker。例如现代 LLM 中的 attention 算子往往具有明显的的数据相关执行时间波动，JIT 能更好地适配这种不均匀性。AOT 的优势则在于它减少了任务真正可运行时的调度路径长度：任务已经在 worker 本地排好队，只要依赖满足就能立即开始，因而有助于降低延迟并减少 scheduler 压力。

MPK 的做法是结合二者，使用混合任务发射。对于那些执行时间稳定、负载容易预测、并且位于关键路径上的任务，MPK 更倾向于使用 AOT，从而减少等待 scheduler 再次介入的开销；对于执行时间波动较大、或需要动态负载均衡的任务，则更适合使用 JIT，以便系统根据运行时状态灵活分配。通过这种组合，MPK 同时获得了 AOT 的低延迟和 JIT 的弹性调度能力。

5.3 运行时优化

本小节介绍一组用于最小化 MPK 运行时开销的优化。

分页共享内存抽象。 在传统 CUDA 或 Triton 编程模型中，shared memory 是 thread block 私有的高速片上存储，并且只在 kernel 生命周期内存在。因此，当前按算子逐核执行的模型通常默认每个 kernel 可以独占整块 shared memory。

但这种独占式设计会阻碍跨任务软件流水，因为后一任务的数据加载阶段往往需要与前一任务的计算阶段重叠，而两者都需要访问 shared memory。为了解决这个问题，MPK 引入了分页共享内存抽象。系统把 shared memory 划分为多个固定大小的页，任务实现则被改写为在这些页面上操作，而不再假设自己独占一整块单体共享内存。任务可以按需获取一个或多个页，在不再需要时释放它们；一旦开始释放页面，就不能再申请新的页面，以保证使用模式单调、便于调度。当当前任务释放页面后，MPK 就能立即把这些可用页面预分配给下一个任务，并启动数据预取。

跨任务软件流水。 为了让同一 worker 上连续执行的任务之间也能形成软件流水 (§ 2.1)，MPK 将每个任务拆成预加载和计算两个阶段。预加载阶段负责把输入张量的一部分从 device memory 拉入 shared memory，并启动任务内部的软件流水，但本身不进行计算。

当满足两个条件时，MPK 会机会式地把当前任务 T_1 的计算阶段与后继任务 T_2 的预加载阶段重叠起来：(1) T_1 已经发出了自己所需的全部数据传输指令；(2) 当前存在足够的 shared-memory 页面供 T_2 预加载使用。这样的流水不会破坏 T_1 的执行正确性，因为 MPK 会插入适当的 SM 内同步屏障，确保 T_2 的内存传输不会与 T_1 尚未完成的数据搬运冲突。

任务描述预取。 每个 worker 都在 device memory 中维护 JIT 和 AOT 两类任务队列。每个任务还对应一个描述符，用于记录输入张量、输出张量及其配置参数；在当前实现中，一个任务描述占 352 字节（见 § 6.1）。为了降低任务入队与出队延迟，并隐藏读取 device memory 的成本，MPK 使用一个轻量级预取机制，在真正需要之前就把后续任务的描述符拉入 shared memory。

6 实验评估

本节围绕三个问题评估 MPK。首先，§ 6.3 将 MPK 的 mega-kernel 执行模型与最先进的按算子逐核执行系统进行对比。其次，§ 6.4 分析 MPK 在大规模 DNN 多 GPU 执行中的可扩展性与效率。最后，§ 6.5 研究 MPK 中各项优化对整体性能提升的贡献。

我们将评估重点放在 LLM serving 上，原因有二。其一，这一场景已经存在 SGLang 与 vLLM [22, 37] 等经过充分优化的按算子逐核执行基线，因此非常适合检验 MPK 的优势。其二，LLM serving 天然具有动态性：不同迭代中的 batch size、sequence length 以及 prefill/decode 的比例都可能发生显著变化，这种异构工作负载同时考验编译器与运行时。需要说明的是，MPK 的编译器与运行时本身并不依赖具体模型，也可以支持任意 DNN 架构。

6.1 实现细节

我们将 MPK 实现为 PyTorch 的一个 kernel backend。用户只需在 PyTorch 的编译接口中指定 MPK 作为后端，即可通过 `torch.compile(backend=MPK)` 把程序编译

Table 1: MPK 在实验中的配置。

GPU	SM 数量	Worker 数量	Scheduler 数量
A100	108	104	16
H100	132	128	16
B200	148	144	16

成一个 MPK mega-kernel。该调用会触发 MPK 编译器生成 mega-kernel，并将其作为可调用的 PyTorch 函数返回；执行该函数时，只会发起一次 mega-kernel 启动。当前 MPK 的实现包含约 4 万行 C++、8.4 万行 CUDA 和 1 万行 Python 代码。核内并行运行时由 CUDA 实现，并使用 device memory 中的 semaphore 来协调 worker 与 scheduler。MPK 编译器使用 C++ 和 Python 编写，可自动将输入张量程序转换为面向特定 GPU 类型优化后的 tGraph。对于计算任务，编译器会集成 Mirage superoptimizer [32] 自动生成高性能 CUDA 实现；对于核内跨 GPU 通信，则直接调用 NVSHMEM [9]。

我们的实现还包含若干关键优化，用于降低运行时开销并高效支持动态工作负载。

任务发射开销。 由于 MPK 会把计算分解为比传统 GPU kernel 更细粒度的任务，因此降低单任务开销对性能至关重要。为此，MPK 采用了多种轻量化实现。首先，运行时中的 worker 与 scheduler 都非常轻量：事件队列和任务队列都用 GPU device memory 中的环形缓冲区实现，入队与出队仅依赖低成本的 `atomicAdd`。其次，MPK 当前采用去中心化调度，每个 scheduler 仅依据本地状态分配任务，从而避免了全局协调所带来的通信和同步开销。尽管当前实现使用的是去中心化策略，运行时本身仍保留了扩展到全局感知或分层调度策略的空间，未来可以进一步探索这些设计及其性能权衡。

支持运行时动态性。 为了展示 MPK 对高度动态负载的支持能力，我们将系统扩展到能够支撑 LLM serving 所需的关键机制，包括 *continuous batching* [35] 与 *paged attention* [22]。在处理任务图的起始事件时，scheduler 会完成一个新的 decoding 迭代准备过程，包括：(1) 移除上一轮已经完成的请求；(2) 接纳新到达的请求；(3) 更新每个请求对应的 KV-cache 元数据。以上逻辑全部封装在单个任务中执行，而 KV-cache 元数据则存储在 device memory 中，供 paged-attention 任务直接访问。

为了适应 LLM serving 中固有的动态 batch size，MPK 会为若干具有代表性的 batch size 生成多份专门优化的 tGraph（具体为从 1 到最大 batch 的各个 2 的幂）。运行时根据当前 batch size 选择合适的任务图执行。这样既能让编译器针对特定 batch size 做激进优化，又能保持对动态工作负载的适应能力。

6.2 实验设置

我们在五个广泛部署的 LLM 上评估 MPK，这些模型覆盖了不同参数规模与架构家族，同时测试三代 NVIDIA GPU：A100、H100 与 B200。Table 1 总结了每种 GPU 上

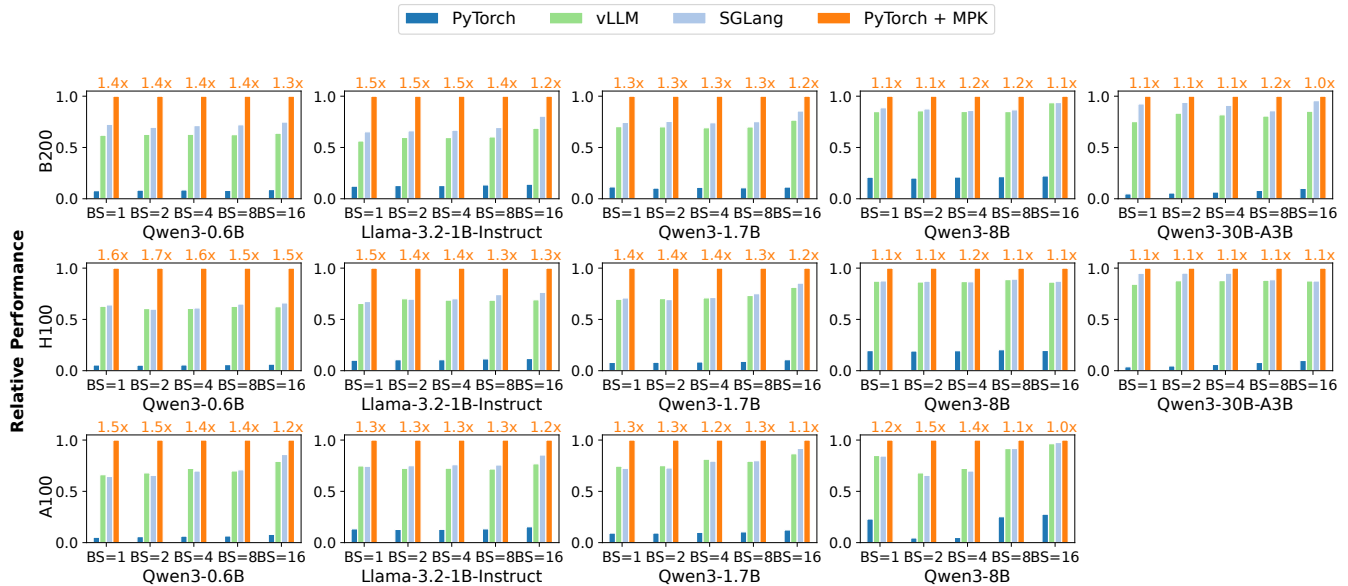


Figure 8: 在 A100、H100 和 B200 上，比较 MPK 与现有系统在五个模型上的性能。所有结果均以 MPK 归一化（越高越好）。每个 MPK 柱子上方的数字表示它相对于当前最优基线的加速比。Qwen3-30B-A3B 在 A100 上的结果被省略，因为该模型无法放入单张 A100。

的 MPK 配置。所有实验中，我们固定保留 4 个 SM 给 scheduler，因此共有 16 个 scheduler warp；其余 SM 则分配给 worker。所有 GPU 的 shared-memory page size 都固定为 32KB，对应在 A100、H100 与 B200 上分别拥有 5、7 和 7 个页面。

为了排除请求到达模式的干扰，所有实验均采用离线的批量推理设置，并测试不同的最大 batch size。每个请求都使用固定长度为 64 的 prompt，并生成 1024 个 token。这样可以避免因请求并发不足导致的服务端阻塞，从而更干净地比较由 MPK 带来的系统级性能收益。

6.3 端到端结果

我们首先把 MPK 与 SGLang 和 vLLM 这两个最先进的 LLM serving 系统进行端到端比较。后两者都采用按算子逐核执行的方法，并依赖大量专用 kernel 库，包括 FlashInfer [34] 与 FlashAttention [7] 实现高性能 attention，cuBLAS 与 cuTLASS [2] 实现矩阵乘法，以及 CUDA 或 Triton [28] 实现其他算子。所有系统都通过 HuggingFace Transformers [6] 加载模型，使用 bfloat16，并启用 paged attention [22] 与 continuous batching [35]。

MPK 与这些系统的核心差异在于：MPK 将 page allocation 和 request scheduling 也直接融合进 mega-kernel 内部，而 SGLang 与 vLLM 仍在 CPU 侧执行这些工作，因此会引入额外的 host-device 同步与分发开销。实验表明，在 B200、H100 与 A100 上，面对从 batch size 1 到 16 的不同设置，MPK 都能在多个模型上稳定超过现有系统。整体来看，MPK 相对最优现有方案取得了 1.0–1.7 $\times$ 的加速，并把 LLM serving 性能进一步逼近硬件极限。

对于 MoE 模型，MPK 还针对 token 路由带来的额外预

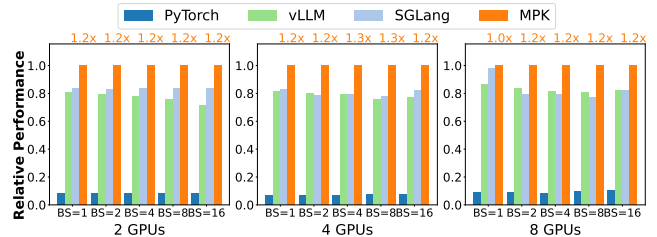


Figure 9: 在 tensor parallelism 下，比较 MPK 与现有系统在多张 H100 上执行 Qwen3-1.7B 推理的性能。所有系统的结果均以 MPK 归一化（越高越好）。

处理做了专门优化。以 SGLang 为例，路由到同一 expert 的 token 需要先被打包成连续布局；在 Qwen3-30B-A3B、batch size 为 1 的情况下，这一步最多可占到总 MoE 执行时间的 11%。而在 MPK 中，如果额外引入独立预处理任务，反而会抬高调度开销。因此，MPK 将 token 级 gather 直接融合进 GEMM 任务的数据加载阶段，用异步拷贝替代独立 gather kernel，从而在不增加调度点的前提下完成高效数据搬运，并持续优于 SGLang 的实现。

6.4 多 GPU 结果

我们进一步在 NVIDIA H100 DGX 上评估 MPK 跨多 GPU 的扩展性。与对比基线一样，这里采用 Megatron-LM [26] 中提出的 *tensor model parallelism*。用户只需在 attention 和 gated MLP 模块后插入 AllReduce 层，MPK 就会自动把这些 collective operator 编译为两类任务：(1) 基

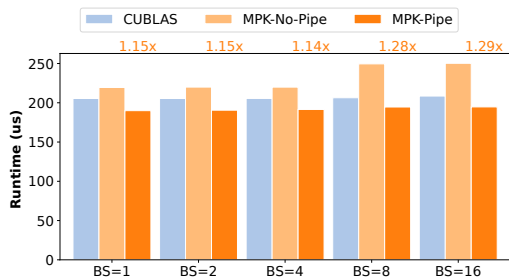


Figure 10: 跨任务流水的消融实验。我们在 NVIDIA B200 上测量 Qwen3-8B 最后一个线性层的运行时间，纵轴为实际耗时（微秒，越低越好），数字表示 MPK-Pipe 相对于 MPK-No-Pipe 的加速比。

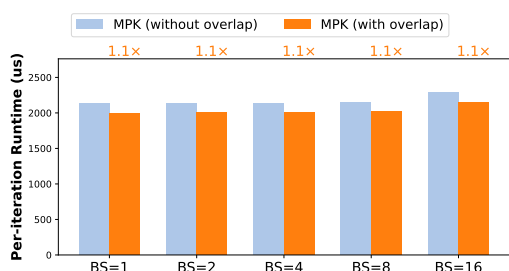


Figure 11: 计算-通信重叠的消融实验。我们在 4 张 H100 上、采用 tensor parallelism 运行 Qwen3-1.7B，比较有无计算-通信重叠时的单次迭代耗时（越低越好）。

于 NVSHMEM `nvshmem_signal_wait_until` 的跨 GPU 数据传输任务；(2) 本地归约任务。这样一来，同步的 collective 通信就被转换成完全异步的任务，可自然融入 MPK 的任务化、事件驱动运行时。

Figure 9 展示了结果。与直接使用手工优化 kernel、CUDA Graphs 和 `torch.compile` 的 PyTorch 相比，MPK 的 mega-kernel 执行可将吞吐提升最高 10 $\times$ 。即便相对于 SGLang 与 vLLM 这类已经高度优化的服务系统，MPK 在扩展到 8 张 H100 时仍可取得 1.1–1.4 $\times$ 的加速。这些收益主要来自三个基线系统缺失的优化：其一，MPK 在 mega-kernel 内完成 page allocation 与 request scheduling，消除了 CPU 侧分发开销；其二，MPK 的异步执行模型能够重叠计算任务与 collective 通信；其三，MPK 消除了 kernel barrier，并支持跨任务软件流水。后两项优化的影响会在下文消融实验中进一步分析。

6.5 消融实验

本节评估 MPK 支持的两项关键优化：跨任务流水与计算-通信重叠。

跨任务流水。 如 § 5.3 所述，MPK 能在执行当前任务计算的同时，为下一个任务预加载输入张量块。Figure 10 展示了在 Qwen3-8B 最后线性层上的消融结果。启用跨任务流水后，单任务运行时间降低了 1.2–1.3 $\times$ ，甚至超过了 cuBLAS 编译 kernel 的性能。

计算-通信重叠。 MPK 通过细粒度任务依赖建模（见 Figure 5），使运行时能够机会式重叠计算任务与通信任务。为了评估其作用，我们可以退化回粗粒度依赖表示，即只用单个事件表示 collective operator（如 AllReduce）与之前任务之间的依赖关系，等价于 Figure 5(c) 中的做法。Figure 11 显示，启用计算-通信重叠后，单次迭代延迟还能再下降约 1.1 $\times$ 。

7 相关工作

手工设计的 kernel。 现有 ML 框架，如 TensorFlow XLA [1, 11]、PyTorch [24] 与 TensorRT [27]，大多采用按算子逐核执行的方式，并依赖 GPU 专家为各个独立算子手工设计与实现 kernel。以 attention 为例，已经出现了大量针对特定架构特征或使用场景的专用实现，包括 FlashAttention [5, 16, 19]、FasterTransformer [3] 与 FlashInfer [34]。当前系统建立在一个高度碎片化的专用 kernel 生态上，这使得把整个推理流水线统一到单个、整体性的 mega-kernel 中变得非常困难。

ML 编译器。 大量工作研究了如何通过编译器为张量程序自动生成高性能 kernel。TVM [14, 15]、Anso [36]、Triton [28] 以及其他系统 [17, 18, 20, 39]，都建立在 Halide [23, 25] 所提出的 algorithm-schedule 分离思想之上。另一条研究路线则采用 *superoptimization*，从高层规格中自动搜索高效实现 [21, 30, 31, 33, 38]。不过，这些编译器的核心目标仍是算子级优化，并不支持生成统一的 mega-kernel，也无法协调跨算子的整体执行。

Mega-kernel。 已有的 mega-kernel 研究基本都依赖手工设计。例如，FlashDMoE 将 mixture-of-experts 计算和跨 GPU 通信融合进单个手写 mega-kernel [12]；Spector 等人则为 LLaMA-1B 手工开发了一个低延迟 mega-kernel [8, 29]。这类方案需要大量工程投入和深入的 GPU 专业知识，而且通常无法跨模型、跨 GPU 泛化。相比之下，MPK 提供了一条编译器驱动的自动化路径，能够把张量程序直接转换为高度优化的 mega-kernel，从而无需手工融合或手写 mega-kernel 实现。

8 结论

本文提出了 MPK，这是首个能够将多 GPU 模型推理自动转换为完全融合 mega-kernel 的编译器与运行时系统。通过引入 SM 级任务图与核内并行运行时，MPK 克服了按算子逐核执行模型的根本局限，使跨算子软件流水、计算与通信的细粒度重叠，以及内核启动与 CPU 端调度开销的消除成为可能。实验结果表明，MPK 能将 LLM 服务延迟逼近硬件极限，并在不同模型与 GPU 代际上显著提升吞吐。通过在保持现有 ML 框架可用性的同时，将执行统一到单个 mega-kernel 内，MPK 为构建高性能、编译器驱动的推理系统开辟了一条新路径。

致谢

本研究部分得到 NSF 项目 CNS-2211882 和 CNS-2239351、Sloan 研究奖学金, 以及来自 Amazon、Cisco、Google、Meta、NVIDIA、Oracle、Qualcomm 和 Samsung 的研究资助支持。

References

- [1] Xla: Optimizing compiler for tensorflow. <https://www.tensorflow.org/xla>, 2017. 9
- [2] Nvidia/cutlass: Cuda templates for linear algebra sub-routines. <https://github.com/NVIDIA/cutlass>, 2019. 8
- [3] Transformer related optimizations. <https://github.com/NVIDIA/FasterTransformer>, 2020. 9
- [4] Nvidia nccl. <https://developer.nvidia.com/nccl>, 2021. 1
- [5] Flash-decoding for long-context inference. <https://crfm.stanford.edu/2023/10/12/flashdecoding.html>, 2023. 9
- [6] Huggingface Models. <https://huggingface.co/models>, 2023. 8
- [7] A Triton implementation of the FlashAttention2 algorithm. <https://triton-lang.org/main/getting-started/tutorials/06-fused-attention.html>, 2023. 8
- [8] Designing a Low-Latency Megakernel for Llama-1B. <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>, 2025. 3, 9
- [9] NVIDIA OpenSHMEM Library (NVSHMEM) Documentation. <https://docs.nvidia.com/nvshmem/api/index.html>, 2025. 1, 7
- [10] Programmatic Dependent Launch. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>, 2025. 1
- [11] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. Tensorflow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, OSDI, 2016. 9
- [12] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. Flashdmoe: Fast distributed moe in a single kernel, 2025. 3, 9
- [13] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. 3
- [14] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: end-to-end optimization stack for deep learning. *CoRR*, abs/1802.04799, 2018. 1, 3, 9
- [15] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems 31*, NeurIPS’18. 2018. 9
- [16] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. Flash-decoding for long-context inference, 2023. 1, 9
- [17] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. Tensorir: An abstraction for automatic tensorized program optimization, 2022. 9
- [18] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. Graphene: An ir for optimized tensor computations on gpus. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, page 302–313, New York, NY, USA, 2023. Association for Computing Machinery. 9
- [19] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. Flashdecoding++: Faster large language model inference on gpus, 2024. 9
- [20] Muyan Hu, Ashwin Venkatram, Shreyashri Biswas, Balamurugan Marimuthu, Bohan Hou, Gabriele Oliaro, Haojie Wang, Liyan Zheng, Xupeng Miao, Jidong Zhai, and Zhihao Jia. Optimal kernel orchestration for tensor programs with korch. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS ’24, page 755–769, New York, NY, USA, 2024. Association for Computing Machinery. 9

- [21] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP '19*, page 47 – 62, New York, NY, USA, 2019. Association for Computing Machinery. 3, 9
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Liu, Xin Wu, Michael Zeng, Xiangru Zhang, Yuhao Zou, and Scott Moritz. Efficient memory management for large language models. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023. 7, 8
- [23] Ravi Teja Mullapudi, Andrew Adams, Dillon Sharlet, Jonathan Ragan-Kelley, and Kayvon Fatahalian. Automatically scheduling halide image processing pipelines. *ACM Trans. Graph.*, 35(4), 2016. 9
- [24] Tensors and Dynamic neural networks in Python with strong GPU acceleration. <https://pytorch.org>, 2017. 1, 3, 9
- [25] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '13*, 2013. 9
- [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019. 8
- [27] NVIDIA TensorRT: Programmable inference accelerator. <https://developer.nvidia.com/tensorrt>, 2017. 9
- [28] Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL 2019*, page 10 – 19, New York, NY, USA, 2019. Association for Computing Machinery. 1, 8, 9
- [29] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 3, 9
- [30] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. Unity: Accelerating DNN training through joint optimization of algebraic transformations and parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 267–284. USENIX Association, 2022. 9
- [31] Haojie Wang, Jidong Zhai, Mingyu Gao, Zixuan Ma, Shizhi Tang, Liyan Zheng, Yuanzhi Li, Kaiyuan Rong, Yuanyong Chen, and Zhihao Jia. PET: Optimizing tensor programs with partially equivalent transformations and automated corrections. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 37–54. USENIX Association, July 2021. 9
- [32] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Man Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. Mirage: A {Multi-Level} superoptimizer for tensor programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 21–38, 2025. 1, 2, 3, 5, 7
- [33] Yichen Yang, Phitchaya Phothilimthana, Yisu Wang, Max Willsey, Sudip Roy, and Jacques Pienaar. Equality Saturation for Tensor Graph Superoptimization. *Proceedings of Machine Learning and Systems*, 3:255–268, March 2021. 9
- [34] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for llm inference serving, 2025. 1, 8, 9
- [35] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022. 7, 8
- [36] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor : Generating high-performance tensor programs for deep learning. *CoRR*, abs/2006.06762, 2020. 9

- [37] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24*, Red Hook, NY, USA, 2024. Curran Associates Inc. 7
- [38] Liyan Zheng, Haojie Wang, Jidong Zhai, Muyan Hu, Zixuan Ma, Tuowei Wang, Shuhong Huang, Xupeng Miao, Shizhi Tang, Kezhao Huang, and Zhihao Jia. EINNET: Optimizing tensor programs with Derivation-Based transformations. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 739–755, Boston, MA, July 2023. USENIX Association. 9
- [39] Size Zheng, Yun Liang, Shuo Wang, Renze Chen, and Kaiwen Sheng. Flextensor: An automatic schedule exploration and optimization framework for tensor computation on heterogeneous system. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '20*, page 859–873, New York, NY, USA, 2020. Association for Computing Machinery. 9
- [40] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. {NanoFlow}: Towards optimal large language model serving throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 749–765, 2025. 3